

ViewData Description in Bullet Points

🏢 **Purpose:** ViewData is a dictionary object provided by ASP.NET MVC to pass data from the controller to the view.

🏢 **Type:** It is of type `ViewDataDictionary`, which stores data in key-value pairs.

🏢 **Usage:** Data stored in ViewData is accessible only for the current request. It does not persist across redirects or postbacks.

🏢 **Dynamic Typing:** It is loosely typed; you must cast data to the appropriate type before using it in the view.

🏢 **Advantages:**

- Useful for passing small amounts of data.
- Works without requiring a strongly typed view model.

🏢 **Drawbacks:**

- Data is not type-safe, increasing the chance of runtime errors.
- Less suitable for large or complex data structures compared to `ViewBag` or `Model`.

Code Example: Using ViewData

```
public class HomeController : Controller
{
    public ActionResult Index()
    {
        // Adding data to ViewData
        ViewData["Title"] = "Welcome to ASP.NET MVC";
        ViewData["Items"] = new List<string> { "Item 1", "Item 2", "Item 3" };
    };

    return View();
}
```

```
<!DOCTYPE html>

<html>

<head>

    <title>@ViewData["Title"]</title>

</head>
```

```
<body>

    <h1>@ViewData["Title"]</h1>

    <ul>

        @if (ViewData["Items"] is List<string> items)
        {
            foreach (var item in items)
            {
                <li>@item</li>
            }
        }

    </ul>

</body>

</html>
```

Code Example: Passing and Displaying Different Data Types

```
public class ProductController : Controller
{
    public ActionResult Details()
    {
        ViewData["ProductName"] = "Laptop";
        ViewData["Price"] = 1500.75;
        ViewData["IsAvailable"] = true;
        return View();
    }
}
```

```
<!DOCTYPE html>

<html>

<head>

    <title>Product Details</title>

</head>

<body>

    <h1>Product Details</h1>

    <p><strong>Product Name:</strong> @ViewData["ProductName"]</p>

    <p><strong>Price:</strong> $@ViewData["Price"]</p>

    <p>

        <strong>Availability:</strong>

        @if (ViewData["IsAvailable"] is bool isAvailable && isAvailable)
        {
            <span>In Stock</span>
        }
        else
        {
            <span>Out of Stock</span>
        }
    </p>

</body>

</html>
```

Key Notes

1. `ViewData` is suitable for simple and temporary data-sharing scenarios between the controller and the view.
 2. Always validate the type when accessing `ViewData` values in the view to prevent runtime errors.
 3. For better type safety and maintainability, consider using a **ViewModel** or `ViewBag` instead of `ViewData` when working with large datasets.
-

ViewBag Description in Bullet Points

- **Purpose:** `ViewBag` is a dynamic object used to pass data from the controller to the view in ASP.NET MVC.
- **Type:** It is a wrapper around `ViewData`, leveraging `Dynamic` features introduced in C# 4.0.
- **Dynamic Typing:** Unlike `ViewData`, you don't need to cast the data explicitly; it is dynamically typed.
- **Usage:**
 - Stores data as properties rather than key-value pairs.
 - Accessible only for the current request; data does not persist across redirects or postbacks.
- **Advantages:**
 - Simpler and cleaner syntax compared to `ViewData`.
 - Eliminates the need for explicit casting.
- **Drawbacks:**
 - Not type-safe; runtime errors may occur if you access nonexistent properties.
 - Less suitable for complex or large data structures compared to strongly typed models.
- **Common Use Case:** Best for passing temporary or small amounts of data, like messages or titles, to the view.

Code Example 1: Passing Simple Data with ViewBag

```
public class HomeController : Controller
{
    public ActionResult Index()
    {
        // Adding data to ViewBag
        ViewBag.Title = "Welcome to ASP.NET MVC";
        ViewBag.Message = "This is a dynamic message using ViewBag.";

        return View();
    }
}
```

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>@ViewBag.Title</title>
</head>
<body>
    <h1>@ViewBag.Title</h1>
    <p>@ViewBag.Message</p>
</body>
</html>
```

Code Example 2: Passing Complex Data with ViewBag

```
public class ProductController : Controller
{
    public ActionResult Details()
    {
        ViewBag.ProductName = "Laptop";
        ViewBag.Price = 1500.75;
        ViewBag.Features = new List<string> { "16GB RAM", "512GB SSD",
        "Intel i7 Processor" };

        return View();
    }
}
```

```
<!DOCTYPE html>
<html>
<head>
    <title>Product Details</title>
</head>
<body>
    <h1>Product Details</h1>
    <p><strong>Product Name:</strong> @ViewBag.ProductName</p>
    <p><strong>Price:</strong> $@ViewBag.Price</p>
    <h3>Features:</h3>
    <ul>
        @foreach (var feature in ViewBag.Features)
        {
            <li>@feature</li>
        }
    </ul>
</body>
</html>
```

Code Example 3: Conditional Logic with ViewBag

```
public class AccountController : Controller
{
    public ActionResult Status()
```

```
{
    ViewBag.IsLoggedIn = true;
    ViewBag.UserName = "John Doe";

    return View();
}
```

```
<!DOCTYPE html>
<html>
<head>
    <title>Account Status</title>
</head>
<body>
    <h1>Account Status</h1>
    @if (ViewBag.IsLoggedIn)
    {
        <p>Welcome, @ViewBag.UserName!</p>
    }
    else
    {
        <p>Please log in to access your account.</p>
    }
</body>
</html>
```

Key Notes

1. **Dynamic Nature:** The flexibility of `ViewBag` makes it simpler to use than `ViewData`, but its lack of type safety can lead to errors.
 2. **Temporary Scope:** Like `ViewData`, `ViewBag` is valid only for the current request.
 3. **Readable Syntax:** The property-based syntax (`ViewBag.Property`) is more intuitive and concise than the key-value syntax of `ViewData`.
 4. **For Larger Models:** Use strongly typed view models for complex data rather than relying on `ViewBag`.
-

TempData Description in Bullet Points

- **Purpose:** `TempData` is used to pass data between two successive requests in ASP.NET MVC.
- **Type:** It is of type `TempDataDictionary` and works like a temporary storage for key-value pairs.
- **Usage:**
 - Data persists only for the duration of a single redirect or postback.
 - Ideal for passing small pieces of data like status messages, alerts, or flags.
- **Behavior:**
 - Data stored in `TempData` is removed after it is read.
 - Data remains available until accessed (using `TempData.Peek()` prevents removal).
- **Advantages:**
 - Useful for redirect scenarios where `ViewBag` and `ViewData` do not persist.

- Simplifies passing temporary data between controllers.
- **Drawbacks:**
 - Not type-safe; casting may be required when accessing stored data.
 - Can introduce complexity if overused for large or complex data.

Code Example 1: Simple Data Transfer with TempData

```
public class HomeController : Controller
{
    public ActionResult SetTempData()
    {
        TempData["Message"] = "This is a temporary message.";
        return RedirectToAction("DisplayTempData");
    }

    public ActionResult DisplayTempData()
    {
        return View();
    }
}
```

```
<!DOCTYPE html>
<html>
<head>
    <title>TempData Example</title>
</head>
<body>
    <h1>TempData Example</h1>
    @if (TempData["Message"] != null)
    {
        <p>@TempData["Message"]</p>
    }
    else
    {
        <p>No TempData available.</p>
    }
</body>
</html>
```

Code Example 2: Using TempData.Peek() to Preserve Data

```
public class ProductController : Controller
{
    public ActionResult SetTempData()
    {
        TempData["Notification"] = "Product successfully added!";
        return RedirectToAction("ShowNotification");
    }
}
```

```

    public ActionResult ShowNotification()
    {
        return View();
    }

    public ActionResult ConfirmNotification()
    {
        var notification = TempData.Peek("Notification");
        return View((object)notification);
    }
}

```

```

<!DOCTYPE html>
<html>
<head>
    <title>Show Notification</title>
</head>
<body>
    <h1>Notification</h1>
    @if (TempData["Notification"] != null)
    {
        <p>@TempData["Notification"]</p>
        <a href="@Url.Action("ConfirmNotification", "Product")">Confirm</a>
    }
</body>
</html>

```

```

<!DOCTYPE html>
<html>
<head>
    <title>Confirm Notification</title>
</head>
<body>
    <h1>Confirm Notification</h1>
    <p>@Model</p>
</body>
</html>

```

Code Example 3: Passing Complex Data with TempData

```

public class OrderController : Controller
{
    public ActionResult PlaceOrder()
    {
        var orderDetails = new { OrderId = 123, Status = "Processed" };
    }
}

```



```

        TempData["OrderDetails"] = orderDetails;
        return RedirectToAction("OrderSummary");
    }

    public ActionResult OrderSummary()
    {
        return View();
    }
}

```

```

<!DOCTYPE html>
<html>
<head>
    <title>Order Summary</title>
</head>
<body>
    <h1>Order Summary</h1>
    @if (TempData["OrderDetails"] is dynamic order)
    {
        <p>Order ID: @order.OrderId</p>
        <p>Status: @order.Status</p>
    }
    else
    {
        <p>No order details available.</p>
    }
</body>
</html>

```

Key Notes

1. **Scope:** `TempData` is best used for scenarios requiring data persistence across redirects or postbacks.
2. **Data Retrieval:**
 - Data is removed after being accessed, unless `Peek()` is used.
 - Always validate and cast data as needed.
3. **Avoid Overuse:**
 - For complex or large data structures, consider using `Session` or a database.
4. **Complementary Use:** Use `TempData` with lightweight, short-lived data like notifications or flags.

Comparison of ViewData, ViewBag, and TempData

Feature	ViewData	ViewBag	TempData
Definition	A dictionary object used to pass data from the controller to the view.	A dynamic object used to pass data from the controller to the view.	A dictionary object used to pass data across requests (e.g., between controllers).
Type	ViewDataDictionary	Dynamic	TempDataDictionary

Data Access	Accessed using keys, e.g., ViewData["Key"].	Accessed using properties, e.g., ViewBag.Key.	Accessed using keys, e.g., TempData["Key"].
Casting	Requires explicit typecasting when accessing data.	No typecasting required due to dynamic nature.	Requires explicit typecasting when accessing data.
Data Persistence	Available only during the current request.	Available only during the current request.	Persists across a single redirect or postback.
Use Case	Suitable for passing simple data like strings, numbers, or small collections.	Ideal for simple data that doesn't require type safety.	Suitable for temporary data that needs to persist between requests, such as notifications or status messages.
Type Safety	Not type-safe; prone to runtime errors if incorrect type is accessed.	Not type-safe; dynamic typing can lead to runtime errors.	Not type-safe; prone to errors if incorrect type is accessed.
Common Usage	Sharing data between controller and view within the same request.	Sharing data between controller and view with cleaner syntax.	Passing temporary data during redirects or postbacks.
Performance	Slightly better than ViewBag due to static typing.	Slower than ViewData due to dynamic typing.	Similar to ViewData but incurs overhead for persisting data across requests.

ViewData Example

```
csharp
Copy code
// Controller
ViewData["Message"] = "Welcome to ASP.NET MVC";

// View
<h1>@ViewData["Message"]</h1>
```

ViewBag Example

```
csharp
Copy code
// Controller
ViewBag.Message = "Welcome to ASP.NET MVC";

// View
<h1>@ViewBag.Message</h1>
```

TempData Example

```
csharp
Copy code
// Controller
```

```
TempData["Message"] = "Data passed between requests.";
return RedirectToAction("NextAction");

// View (NextAction.cshtml)
<h1>@TempData["Message"]</h1>
```

Session Description in Bullet Points

- **Purpose:** Sessions store user-specific data across multiple requests within a web application.
- **Scope:** Data persists throughout the user's session until it expires, the user logs out, or the session is cleared explicitly.
- **Type:** The session object is of type `HttpSessionState`, which allows storing data as key-value pairs.
- **Lifetime:**
 - A session starts when a user accesses the application for the first time.
 - By default, a session ends after 20 minutes of inactivity or when explicitly abandoned.
- **Advantages:**
 - Data persists across requests, unlike `ViewData` and `ViewBag`.
 - Ideal for storing user-specific information like login credentials or cart data.
- **Drawbacks:**
 - Consumes server memory for every active session.
 - Not suitable for large amounts of data or multi-user data sharing.
- **Security:** Stored on the server, so it's more secure than client-side options like cookies, but still vulnerable to session hijacking if not properly managed.

Code Example 1: Simple Session Usage

```
public class AccountController : Controller
{
    public ActionResult Login()
    {
        // Setting session data
        Session["UserName"] = "JohnDoe";
        Session["IsLoggedIn"] = true;

        return RedirectToAction("Dashboard");
    }

    public ActionResult Dashboard()
    {
        return View();
    }
}
```

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>Dashboard</title>
</head>
<body>
  <h1>Welcome to the Dashboard</h1>
  @if (Session["IsLoggedIn"] != null && (bool)Session["IsLoggedIn"])
  {
    <p>Hello, @Session["UserName"]!</p>
  }
  else
  {
    <p>Please log in to access the dashboard.</p>
  }
</body>
</html>
```

```
public class AccountController : Controller
{
  public ActionResult Logout()
  {
    // Clearing the session
    Session.Clear();
    return RedirectToAction("Login");
  }

  public ActionResult Login()
  {
    return View();
  }
}
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>
  <h1>Login</h1>
  <form action="@Url.Action("Login", "Account")" method="post">
    <button type="submit">Log In</button>
  </form>
  @if (Session["UserName"] == null)
  {
    <p>You are logged out.</p>
  }
```

```
}  
</body>  
</html>
```

```
public class CartController : Controller  
{  
    public ActionResult AddToCart()  
    {  
        var cart = new List<string> { "Laptop", "Mouse", "Keyboard" };  
        Session["CartItems"] = cart;  
  
        return RedirectToAction("ViewCart");  
    }  
  
    public ActionResult ViewCart()  
    {  
        return View();  
    }  
}
```

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Shopping Cart</title>  
</head>  
<body>  
    <h1>Your Shopping Cart</h1>  
    <ul>  
        @if (Session["CartItems"] is List<string> cartItems)  
        {  
            foreach (var item in cartItems)  
            {  
                <li>@item</li>  
            }  
        }  
        else  
        {  
            <p>Your cart is empty.</p>  
        }  
    </ul>  
</body>  
</html>
```

Key Notes

1. **Data Persistence:** Sessions persist data across multiple requests, making them ideal for user authentication and state management.
 2. **Security:**
 - Use HTTPS to secure session cookies.
 - Implement timeout mechanisms to prevent unauthorized access.
 3. **Memory Consideration:** Avoid storing large objects in sessions as they consume server resources.
 4. **Session Management:**
 - Use `Session.Clear()` to remove all session data.
 - Use `Session.Abandon()` to end the session explicitly.
-